CHAPTER 3

CONCEPTUAL FRAMEWORK AND TECHNICAL BACKGROUND

3.1 Conceptual Framework

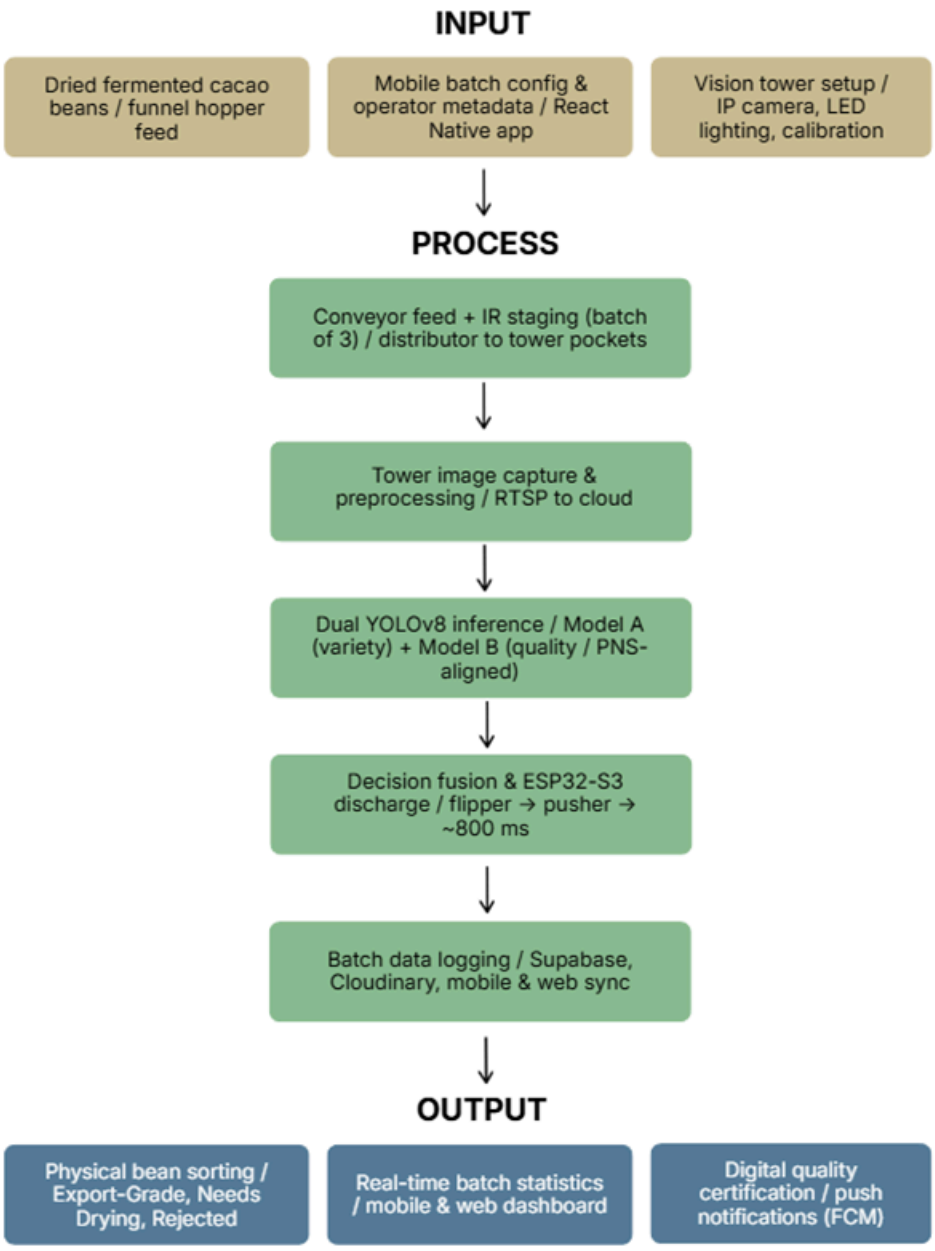

Figure 3.1 Conceptual Framework of CacaoScan

CacaoScan is designed using the Input-Process-Output (IPO) model within a standalone AIoT architecture that combines cloud-hosted dual-model inference with on-site mechanical

staging and discharge. The framework maps the complete data and control cycle from bean infeed through batch staging, tower imaging, cloud YOLOv8 analysis, sequenced push-and-flip sorting, and digital batch documentation. Based on the IPO figure above, the system components are discussed as follows:

Input

The primary input consists of dried fermented cacao beans loaded into the funnel hopper at the infeed module. Secondary inputs include conveyor transport motion from the 12V geared motor, infrared beam-break counts at the staging gate, pocket-position video frames from the Industrial HD IP Camera, mobile batch commands from the React Native application, and trained YOLOv8 model weights deployed to Hugging Face Inference Endpoints for Model A (Criollo, Forastero, Trinitario) and Model B (Export-Grade, Needs Drying, Rejected).

Process

First, the funnel hopper and vibratory feed assist deposit beans onto the V-trough conveyor, which transports them to the staging gate while encouraging lengthwise alignment. Second, the infrared beam-break sensor counts three beans, the ESP32-S3 stops the conveyor through an opto-isolated relay, and the tilting distributor chute places each bean into one of three pockets inside the enclosed vision tower. Third, the Industrial HD IP Camera captures a single controlled image of the pocketed batch and sends it over RTSP and Wi-Fi to the cloud FastAPI service on Hugging Face Inference Endpoints, where Model A and Model B return per-position variety and quality labels. Fourth, decision fusion assigns each pocket a destination bin, with Model B governing export readiness while Model A results are logged for variety analytics. Fifth, the firmware pre-positions the MG996R sorting flipper, fires the corresponding SG90 lateral pusher, and allows approximately 800 milliseconds of rail-clearance time before discharging the next bean. Lastly, when all three beans are sorted, actuators return to home position, the conveyor relay re-enables, and batch records sync to Supabase for mobile and web dashboard updates.

Output

The primary output is physically sorted cacao beans in three designated bins labeled Export-Grade, Needs Drying, and Rejected. Secondary outputs include per-batch and per-bean classification logs, real-time counter updates on the React Native mobile command center and React JS responsive web dashboard, PDF quality certificates, push notifications through Firebase Cloud Messaging, archived tower images in Cloudinary, and persistent quality records in Supabase for cooperative export documentation.

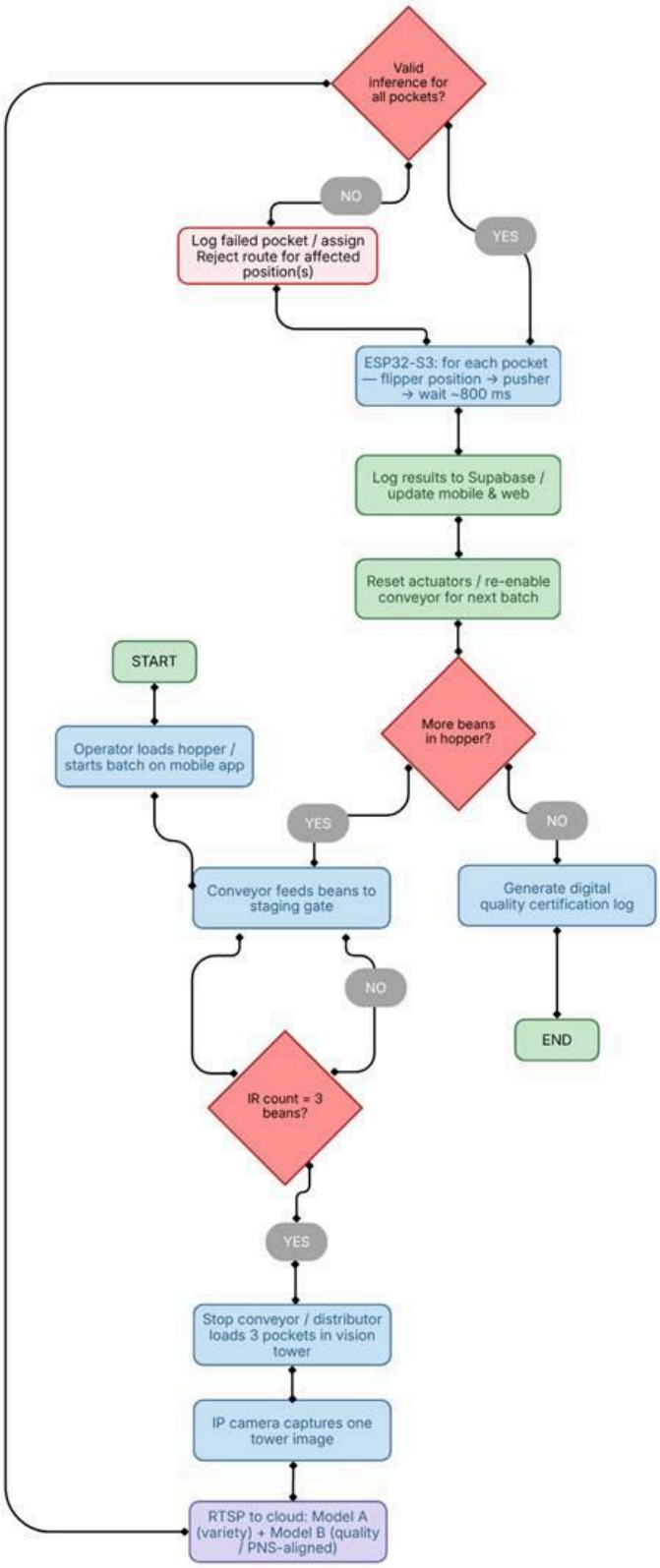

Figure 3.2 CacaoScan Bean Processing Flow Chart

Based on the flowchart above, the process begins when the operator loads dried cacao beans into the hopper and starts a batch session from the React Native mobile application. The conveyor transports beans to the staging gate until the infrared sensor registers three beans, at which point the belt stops and the distributor loads the vision tower pockets. The Industrial HD IP Camera captures one tower image, the cloud service runs Model A for variety and PNS-aligned Model B for quality, and the ESP32-S3 receives three fused sort decisions.

For each pocket position, the sorting flipper moves to the assigned bin path, the corresponding lateral pusher ejects the bean onto the common sorting rail, and the system waits approximately 800 milliseconds for the rail to clear before continuing. If inference fails or an image is unreadable, the affected pocket is logged and routed to the reject path without stopping the remaining batch sequence. When all three beans are discharged, the actuators reset and the conveyor re-enables for the next three-bean cycle until the hopper is empty, after which the system generates the digital quality certification log.

Decision fusion gives priority to Model B quality output when determining the physical bin for each pocket position, while Model A variety results are recorded for batch analytics. Timed rail clearance between push cycles prevents bean collisions on the common sorting rail. Invalid inference or unreadable tower images are logged and handled through the reject routing path without halting the full machine cycle.

3.2 Technical Background

Recent post-harvest cacao grading in the Philippines still relies on manual table sorting and screen-only mobile assessment tools that display quality estimates without physical separation. Industrial automated sorters integrate vision and actuation but remain economically inaccessible to barangay-level cooperatives in the Zamboanga Peninsula. Existing software-only tools therefore leave the most labor-intensive physical sorting step unresolved for farmers in Barangay Ayala, Pasonanca, and surrounding communities.

To address the gap between manual sorting and industrial automation, the Capstone 1 CacaoScan prototype uses a standalone AIoT framework that combines conveyor-fed batch staging, an enclosed vision tower, cloud-hosted dual YOLOv8 inference on Hugging Face Inference Endpoints, and ESP32-S3-controlled multi-actuator discharge with isolated power electronics. Model training and optimization occur on an NVIDIA GPU workstation using Ultralytics YOLOv8, ONNX, and TensorRT before cloud deployment, while the physical machine at the client site operates autonomously over Wi-Fi without an on-site inference laptop.

3.2.1 Hardware Components

The infeed and transport module consists of a funnel hopper with vibratory feed assist, a white matte silicone V-trough conveyor belt driven by a 12V geared motor, and acrylic side walls that maintain bean alignment during transport. Beans exit the conveyor at the staging gate, where

a servo-controlled stop and infrared beam-break sensor form the fixed batch-of-three queue used by the rest of the pipeline.

The staging and distribution module uses a tilting distributor chute mounted on a servo pivot to deposit the queued beans into the left, center, and right pockets of the vision tower tray. The vision tower itself is an enclosed light-isolated structure with black interior panels, a ceiling-mounted Industrial HD IP Camera, and a white LED ring light that provides uniform illumination for stable multi-bean imaging.

The discharge and sorting module contains an SG90 lateral pusher array that ejects beans from each pocket onto a shared inclined sorting rail, and an MG996R sorting flipper at the rail terminus that routes each bean into Export-Grade, Needs Drying, or Rejected bins. The control electronics are housed in a front-mounted transparent IoT enclosure containing the ESP32-S3, separate LM2596 buck converters for logic and servo power rails, an opto-isolated relay for the conveyor motor, and fuse-protected 12V distribution with common ground wiring.

3.2.2 AI Model Training Workflow

The AI model training workflow follows a structured pipeline aligned with the developmental methodology described in Chapter 4. Stage 1 involves image collection from the vision tower and supplemental datasets using the Industrial HD IP Camera, with Roboflow annotation for Model A (Criollo, Forastero, Trinitario) and Model B (Export-Grade, Needs Drying, Rejected), including pocket-position labels for staged multi-bean images. Stage 2 applies dataset partitioning with stratified balancing across 3,000 to 5,000 annotated images drawn from Kaggle, Roboflow, and at least 500 local samples from Barangay Ayala and Pasonanca, using augmentation, resizing to 640 by 640 pixels, and normalization preprocessing.

Stage 3 trains each YOLOv8 Nano variant independently on an NVIDIA GPU workstation with early stopping based on validation metrics. Stage 4 exports optimized weights through ONNX and TensorRT quantization before deployment to Hugging Face Inference Endpoints for cloud serving. Stage 5 conducts integrated system testing to verify that cloud inference returns correct per-pocket labels, the ESP32-S3 executes the flipper-and-pusher sequence in the proper order, and the 150-bean expert-labeled test set achieves the target detection accuracy and mechanical sorting success rate.

3.2.3 Image Acquisition and Vision Tower Standards

Image acquisition standards require each batch of three beans to occupy the left, center, and right pockets of the vision tower tray under uniform LED ring illumination, with the tower enclosure blocking external ambient light that could distort color or defect detection. The Industrial HD IP Camera captures one high-definition tower image per batch cycle, which is transmitted through RTSP and preprocessed to 640 by 640 pixels before dual-model cloud inference. The same tower setup is used during local field sample collection in Barangay Ayala

and Pasonanca, model retraining, and cooperative prototype evaluation to reduce lighting and positional variability.

3.2.4 System Architecture Overview

The CacaoScan system architecture consists of four interconnected layers: the physical machine layer (infeed conveyor, staging gate, vision tower, pushers, sorting rail, flipper, and bins), the cloud inference layer (Hugging Face-hosted FastAPI with dual optimized YOLOv8 models), the IoT control layer (ESP32-S3 firmware, relay switching, and servo sequencing), and the monitoring layer (Node.js API, Supabase database, React Native mobile app, and React JS web dashboard). A detailed system architecture diagram, context diagram, and data flow diagrams are presented in Chapter 4; Figure 3.1 and Figure 3.2 summarize the IPO model and batch-processing flowchart at the technical background level.

3.2.5 Inference-to-Actuation Synchronization and Reliability

Batch-to-actuation synchronization is managed through firmware state logic on the ESP32-S3. After the infrared sensor confirms three beans and the tower image is captured, the controller waits for cloud inference results before executing discharge commands in pocket order. The MG996R flipper moves to the assigned bin path first, the corresponding SG90 pusher fires, and an approximately 800-millisecond clearance interval elapses before the next bean is released so the common sorting rail remains unobstructed.

Mechanical reliability is evaluated using sorting success rate during controlled runs of 150 expert-labeled beans, actuator return-to-home consistency, and batch-cycle completion without conveyor or gate jamming. Separate logic and servo power rails, relay-isolated motor switching, and fuse protection reduce control resets caused by actuator current spikes. Performance targets require detection accuracy above 85 percent, mechanical sorting success rate of 95 percent, and dependable completion of batch-of-three vision-and-discharge cycles during prototype testing.

3.2.6 Software Requirements

Table 3.1 Software and Platform Requirements

Component	Technology	Purpose
Mobile Application	React Native	Start batches, monitor cycles, receive alerts
Web Dashboard	React JS, Tailwind CSS	Management analytics, PDF certificates, trend views

Cloud AI Service	Python FastAPI on Hugging Face	Per-pocket dual YOLOv8 variety and quality inference
Backend API	Node.js, Express.js	Synchronizes cloud results, firmware commands, and logs
Database	Supabase (PostgreSQL)	Stores batch records, pocket labels, and user profiles
Image Storage	Cloudinary API	Archives vision tower batch images
Notifications	Firebase Cloud Messaging	Sends mobile alerts for batch events and errors
Charting	Victory Native	Renders historical quality and variety statistics
Model Optimization	ONNX, TensorRT	Quantizes YOLOv8 weights for cloud deployment
Firmware	Arduino IDE (C++)	ESP32-S3 batch logic, relay control, and servo sequencing
Feed Transport	12V geared motor, V-trough belt	Aligns and delivers beans to staging gate
Power Electronics	LM2596 buck converters, relay, fuse	Isolates logic and actuator power domains

3.2.7 Network Architecture and End Users

CacaoScan operates in real-time standalone mode at the client cooperative facility. The physical machine connects to the local Wi-Fi network to upload tower images and receive cloud classification results without requiring an on-site laptop for inference. The Node.js backend synchronizes Hugging Face inference results, Supabase batch logs, Cloudinary image archives, and Firebase Cloud Messaging alerts to both the React Native mobile command center and the React JS responsive web dashboard.

End users include cacao farmers and cooperative processing staff who load beans into the hopper, start batch sessions from the mobile app, and monitor discharge progress during sorting operations. Cooperative managers use the web dashboard to review quality trends, export PDF certificates, and analyze variety and grade distributions through Victory Native charts on mobile and web analytics views. Certified cacao graders and DA or BFAR experts serve as third-party consultants for ground-truth validation during accuracy testing.

3.3 Summary

In previous post-harvest workflows, accurate variety segregation and export-oriented quality grading required sustained manual inspection that is slow, subjective, and vulnerable to fatigue. The CacaoScan prototype demonstrates that cloud-hosted dual-model YOLOv8 inference combined with enclosed tower imaging and ESP32-sequenced mechanical discharge can execute the complete sense-decide-act pipeline for cacao beans at cooperative scale in batch-of-three cycles.

By deploying inference on Hugging Face Inference Endpoints while keeping conveyor feeding, tower imaging, and bin routing autonomous at the client site, the system provides digital batch records synchronized through Supabase to mobile and web platforms. Controlled tower acquisition, expert-validated testing with 150 labeled beans, and per-pocket evaluation metrics establish the technical foundation further detailed in Chapter 4.

Variety output from Model A is treated as a cooperative sorting aid and batch analytics input rather than a legally definitive genetic certificate, while Model B quality grading supports export preparation workflows for Zamboanga cacao cooperatives.

REFERENCES

[1] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," GitHub, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics. [Accessed: Jun. 19, 2026].

[2] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," IEEE Internet Things J., vol. 3, no. 5, pp. 637-646, Oct. 2016, doi: 10.1109/JIOT.2016.2579198.

[3] L. Atzori, A. Iera, and G. Morabito, "The internet of things: A survey," Comput. Netw., vol. 54, no. 15, pp. 2787-2805, Oct. 2010, doi: 10.1016/j.comnet.2010.05.010.

[4] Philippine Bureau of Agriculture and Fisheries Standards, Philippine National Standard for Cacao Beans, PNS/BAFS 58:2019. Quezon City, Philippines: Department of Agriculture, 2019.

[5] L. dos Santos Cordeiro, I. de Alencar Nääs, and M. Tsuguio Okano, "Smart postharvest management of strawberries: YOLOv8-driven detection of defects, diseases, and maturity," AgriEngineering, vol. 7, no. 8, Art. no. 246, 2025, doi: 10.3390/agriengineering7080246.